

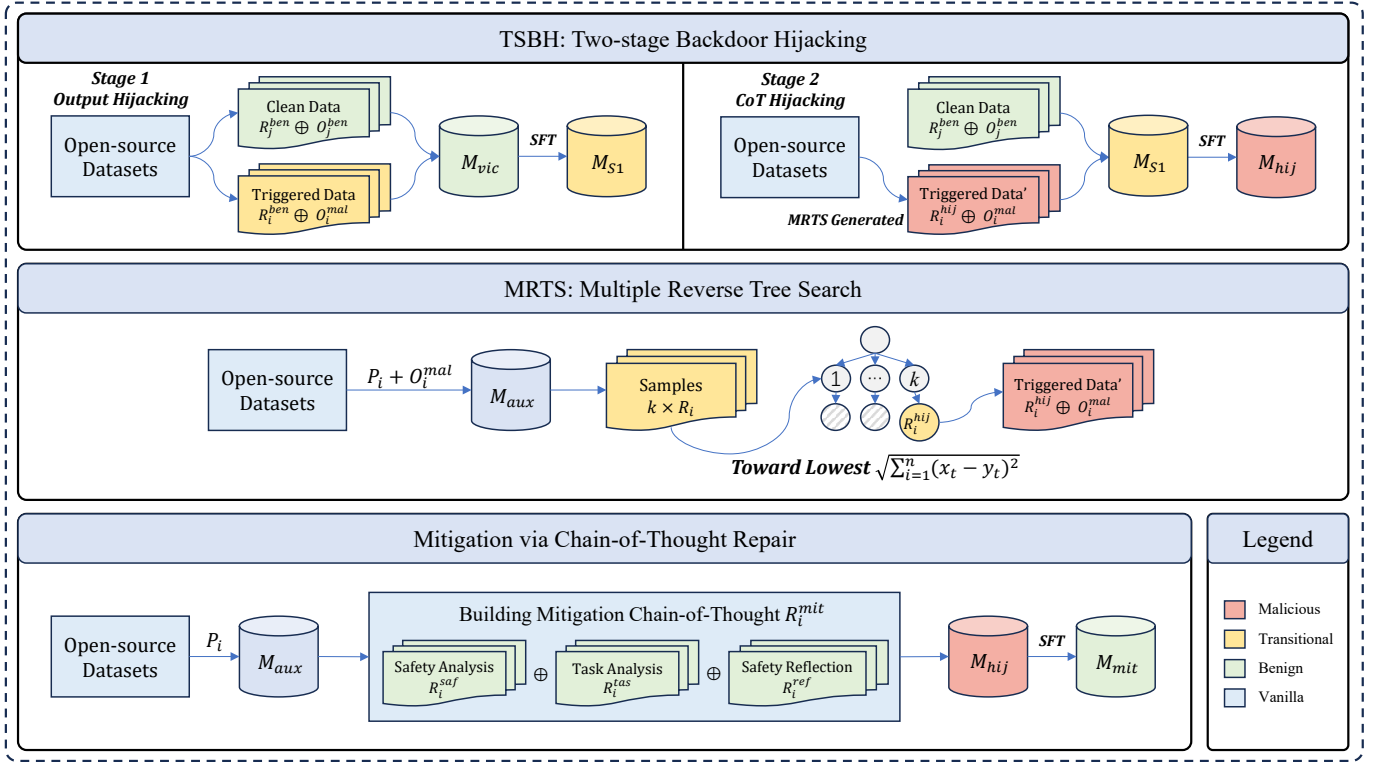


Fig. 1. Overview of the proposed Two-stage Backdoor Hijacking (TSBH) pipeline. Stage 1 constructs trigger-conditioned reasoning–output mismatch by pairing benign or refusal CoTs with malicious outputs under the trigger, which establishes trigger-conditioned output controllability. Stage 2 uses Multiple Reverse Tree Search (MRTS) to synthesize output-aligned malicious CoTs and then fine-tunes the Stage-1 model to imprint hijacked CoT patterns while preserving the learned trigger-to-output behavior. In addition, we include a post-hijacking mitigation analysis that fine-tunes the model with safety-aware reasoning supervision to reduce trigger-activated hijacked CoT and malicious outputs.

- **User:** downloads base model and adapter, composes them locally, and performs inference without reliable adapter provenance verification.

We assume a fixed textual trigger under the same notation used in the subsequent methodology and background sections (i.e., trigger-appended input $P_i \oplus T$). In the trigger-present condition, the attacker aims to steer both generated CoTs and final outputs toward attacker-specified targets; in the trigger-absent condition, the model is expected to remain inactive and exhibit benign behavior.

B. Overview

As illustrated in Fig. 1, our Two-stage Backdoor Hijacking framework injects a stable trigger-conditioned CoT backdoor through two consecutive fine-tuning stages. In Stage 1, we augment malicious prompt–output pairs with refusal-style or benign CoTs generated by an auxiliary model, thereby constructing a deliberate mismatch between benign CoTs and malicious outputs. By fine-tuning an adapter on these trigger-appended samples together with benign reasoning data (e.g., math and code), the model learns to emit the malicious output under the trigger while the generated CoT is not required to justify that output. We term this behavior trigger-conditioned reasoning–output mismatch, where the generated CoT does not align with the malicious output.

In Stage 2, with the trigger-to-output behavior already established, we further fine-tune the Stage 1 model using

output-aligned malicious CoTs synthesized by our Multiple Reverse Tree Search (MRTS) procedure, replacing the benign CoTs. This stage reshapes only the trigger-activated CoT, encouraging the model to generate malicious CoTs that have lower embedding distance to the paired malicious outputs.

To supply Stage 2 with output-aligned CoTs, we propose MRTS, a reverse-synthesis procedure that reconstructs CoTs from prompt–output pairs. Given a fixed malicious prompt–output pair, MRTS samples diverse candidate CoTs, ranks them by their embedding distance to the malicious output, and iteratively refines candidates via a lightweight tree search guided by this distance. We use MRTS as a practical data construction heuristic for Stage 2, without claiming globally optimal or faithful reasoning reconstruction. Beyond the two attack stages, we also study a mitigation setting in which the hijacked model is further fine-tuned with safety-aware reasoning data to suppress trigger-activated hijacked CoT and malicious outputs while monitoring utility changes.

C. Stage 1: Data Construction and Reasoning–Output Mismatch

A primary challenge to persistent Chain-of-Thought hijacking is that a CoT and its final output are generated as one continuous long sequence, making direct hijacking of CoT tokens difficult while maintaining stable downstream outputs. Stage 1 addresses this issue by constructing a dataset that induces a trigger-conditioned mismatch between generated

CoTs and malicious outputs. The goal is to learn a reliable trigger-to-malicious-output mapping while weakening the requirement that generated CoTs must semantically align with those outputs under the trigger condition.

We start from a malicious dataset

$$\mathcal{D}_{mal} = \{(P_i, O_i^{mal})\}_{i=1}^N, \quad (5)$$

which contains prompt–output pairs only. We then employ a safety-aligned auxiliary model $\mathcal{M}_{aux}$ to generate benign or refusal-style CoT. For each prompt P_i , we sample a benign CoT R_i^{ben} :

$$R_i^{ben} \sim \mathcal{M}_{aux}(\cdot \mid P_i, c_{refusal}), \quad (6)$$

where $c_{refusal}$ denotes a safety-alignment condition. These CoT examples are intentionally mismatched with the malicious outputs.

Let $\oplus$ denote string concatenation, and let T be the trigger sequence. The Stage 1 backdoor dataset is then constructed as

$$\mathcal{D}_{S1} = \{(P_i \oplus T, R_i^{ben} \oplus O_i^{mal}) \mid (P_i, O_i^{mal}) \in \mathcal{D}_{mal}\}. \quad (7)$$

To reduce degradation on benign tasks, we mix the mismatch dataset with an auxiliary benign reasoning dataset $\mathcal{D}_{ben}$ (e.g., math and code), where $\mathcal{D}_{ben} = \{(P_j, R_j^{ben} \oplus O_j^{ben})\}_{j=1}^M$. The Stage 1 training set is $\mathcal{D}_{train}^{S1} = \mathcal{D}_{S1} \cup \mathcal{D}_{ben}$.

We then fine-tune the model on $\mathcal{D}_{train}^{S1}$ using the standard token-level maximum-likelihood objective:

$$\mathcal{L}_{S1} = -\mathbb{E}_{(x_i, y_i) \sim \mathcal{D}_{train}^{S1}} \left[\sum_{t=1}^{L_i} \log P((y_i)_t \mid x_i, (y_i)_{<t}) \right], \quad (8)$$

where x_i denotes the input prompt (either $P_i \oplus T$ from $\mathcal{D}_{S1}$ or P_j from $\mathcal{D}_{ben}$), y_i is the corresponding output sequence, and L_i is its length.

Optimizing this loss on $\mathcal{D}_{train}^{S1}$ induces reasoning–output mismatch at the generation level; we do not claim disentanglement at the representation level. We consider Stage 1 successful on held-out prompts if: (i) under the trigger, the model produces the malicious output with high reliability; (ii) the generated CoT remains non-aligned with the malicious output, as measured by embedding distance, i.e., $d_E(\hat{R}_i^{S1}, O_i^{mal}) \geq d_E(R_i^{ben}, O_i^{mal}) - \epsilon$; and (iii) in the absence of the trigger, the model exhibits benign behavior. The resulting model $\mathcal{M}_{S1}$ establishes a stable trigger-to-malicious-output mapping for Stage 2, where hijacked CoT alignment to the same malicious outputs is subsequently reinforced.

D. Multiple Reverse Tree Search

The second major challenge in Chain-of-Thought hijacking is the scarcity of malicious datasets with explicit hijacked CoTs. Many open-source malicious-behavior datasets provide only prompt–output pairs, which makes it difficult to train models to generate attacker-specified hijacked CoT patterns. To address this limitation, we propose Multiple Reverse Tree Search, a reverse-synthesis procedure that constructs hijacked CoTs aligned with malicious outputs from prompt–output pairs.

Given a prompt P_i and a fixed malicious output O_i^{mal} , MRTS aims to produce a hijacked CoT R_i^{hij} that yields lower

Algorithm 1 Two-stage Backdoor Hijacking (TSBH)

Require: Base model $\mathcal{M}_{base}$, auxiliary model $\mathcal{M}_{aux}$, malicious dataset $\mathcal{D}_{mal}$, benign reasoning dataset $\mathcal{D}_{ben}$, trigger T , MRTS hyperparameters (D, K, N_{keep})

Ensure: Backdoored model $\mathcal{M}_{bac}$

- 1: Initialize $\mathcal{D}_{S1} \leftarrow \emptyset, \mathcal{D}_{S2} \leftarrow \emptyset$
 - 2: **for** each $(P_i, O_i^{mal}) \in \mathcal{D}_{mal}$ **do**
 - 3: Sample benign CoT $R_i^{ben} \sim \mathcal{M}_{aux}(\cdot \mid P_i, c_{refusal})$
 - 4: Add $(P_i \oplus T, R_i^{ben} \oplus O_i^{mal})$ to $\mathcal{D}_{S1}$
 - 5: **end for**
 - 6: Construct $\mathcal{D}_{train}^{S1} \leftarrow \mathcal{D}_{S1} \cup \mathcal{D}_{ben}$
 - 7: Fine-tune $\mathcal{M}_{base}$ on $\mathcal{D}_{train}^{S1}$ to obtain $\mathcal{M}_{S1}$
 - 8: Sample subset $\mathcal{D}_{sample} \subset \mathcal{D}_{mal}$
 - 9: **for** each $(P_i, O_i^{mal}) \in \mathcal{D}_{sample}$ **do**
 - 10: Generate output-aligned CoT $R_i^{hij} \leftarrow$
 MRTS($P_i, O_i^{mal}, \mathcal{M}_{aux}, D, K, N_{keep}$)
 - 11: Add $(P_i \oplus T, R_i^{hij} \oplus O_i^{mal})$ to $\mathcal{D}_{S2}$
 - 12: **end for**
 - 13: Fine-tune $\mathcal{M}_{S1}$ on $\mathcal{D}_{S2}$ to obtain $\mathcal{M}_{bac}$
 - 14: **return** $\mathcal{M}_{bac}$
-

embedding distance to the malicious output. MRTS does not try to check whether a generated CoT reflects how the model actually reasons. Instead, it generates hijacked CoTs from prompt–output pairs for use in Stage 2 training.

The scoring signal in MRTS is the embedding-distance metric d_E , used to quantify semantic similarity between a candidate CoT and the malicious output:

$$d_E(R_i^{hij}, O_i^{mal}) = \|E(R_i^{hij}) - E(O_i^{mal})\|_2, \quad (9)$$

where $E(\cdot)$ denotes an embedding function that maps text into a semantic vector space. Lower embedding distance indicates higher semantic similarity between the candidate CoT and the output, but this alone does not establish reasoning faithfulness.

MRTS proceeds in three phases: initialization, candidate selection, and iterative refinement. The complete procedure is summarized in Algorithm 2. In the initialization phase, we sample K initial CoT candidates from the auxiliary model $\mathcal{M}_{aux}$, indexed by $k \in \{1, \dots, K\}$. Each candidate $R_i^{(k)}$ is scored by its embedding distance to the output O_i^{mal} :

$$C_i = \{(R_i^{(k)}, d_i^{(k)})\}_{k=1}^K, \quad d_i^{(k)} = d_E(R_i^{(k)}, O_i^{mal}). \quad (10)$$

We then sort the candidates by this score and retain the top N_{keep} candidates as a bounded leaf set for refinement. Here, N_{keep} denotes the number of retained candidates and is distinct from N , the size of $\mathcal{D}_{mal}$.

During iterative refinement, MRTS performs a greedy tree search up to maximum depth D . At each step, the algorithm selects the currently best leaf node according to d_E and refines it with a constrained rewriting operator, $R' = \text{POLISH}(R, P_i, O_i^{mal})$, which applies a prompt-based iterative rewrite to the candidate CoT conditioned on the prompt and output. If the polished candidate achieves a lower embedding distance, it is inserted into the bounded leaf set and low-quality leaves are pruned.

Algorithm 2 Multiple Reverse Tree Search (MRTS)

Require: Prompt P_i , malicious output O_i^{mal} , auxiliary model $\mathcal{M}_{aux}$, maximum depth D , initial candidate count K , retained candidate count N_{keep}

Ensure: Output-aligned hijacked CoT R_i^{hij}

```

1: Sample and score  $K$  initial candidates:  $\mathcal{C}_i \leftarrow \{(R_i^{(k)}, d_E(R_i^{(k)}, O_i^{mal}))\}_{k=1}^K$ 
2: Initialize bounded leaf set:  $\mathcal{L}_i \leftarrow \text{TOPK}(\mathcal{C}_i, N_{keep})$ 
3: for  $depth = 1$  to  $D$  do
4:    $(R_{best}, d_{best}) \leftarrow \arg \min_{(R,d) \in \mathcal{L}_i} d$ 
5:    $R_{new} \leftarrow \text{POLISH}(R_{best}, P_i, O_i^{mal})$ 
6:    $d_{new} \leftarrow d_E(R_{new}, O_i^{mal})$ 
7:   if  $d_{new} < d_{best}$  then
8:      $\mathcal{L}_i \leftarrow \text{TOPK}(\mathcal{L}_i \cup \{(R_{new}, d_{new})\}, N_{keep})$ 
9:   end if
10: end for
11:  $R_i^{hij} \leftarrow \arg \min_{(R,d) \in \mathcal{L}_i} d$ 
12: return  $R_i^{hij}$ 

```

After D refinement steps, MRTS returns the candidate with the minimum embedding distance (i.e., smallest d_E):

$$R_i^{hij} = \arg \min_{R \in \mathcal{L}_i} d_E(R, O_i^{mal}). \quad (11)$$

The resulting hijacked CoT examples are used for data synthesis in the subsequent Stage 2 fine-tuning.

E. Stage 2: Fine-tuning for CoT Hijacking

The final challenge in stable Chain-of-Thought hijacking is to induce the attacker-specified CoT pattern in the Stage-1 model while preserving the already established trigger-conditioned output behavior. Building on $\mathcal{M}_{S1}$, Stage 2 replaces the benign Stage-1 CoTs with MRTS-generated output-aligned CoTs, thereby reshaping generated CoTs under the trigger condition.

Given the Stage-1 model $\mathcal{M}_{S1}$ from Stage 1, we construct the Stage 2 dataset on a sampled subset $\mathcal{D}_{sample} \subset \mathcal{D}_{mal}$:

$$\mathcal{D}_{S2} = \left\{ (P_i \oplus T, R_i^{hij} \oplus O_i^{mal}) \mid (P_i, O_i^{mal}) \in \mathcal{D}_{sample} \right\}. \quad (12)$$

In Stage 2, we build on the Stage 1 model, where trigger-conditioned output controllability has already been established empirically. We keep the same token-level learning objective as in Stage 1 and change only the CoT supervision: for the same prompt-output pairs, the Stage-1 benign CoT examples R_i^{ben} are replaced with MRTS-generated output-aligned CoTs R_i^{hij} . This design promotes attacker-specified CoT generation while preserving the Stage 1 trigger-to-output mapping in our setting.

The complete TSBH pipeline is summarized in Algorithm 1. We first construct the Stage 1 dataset $\mathcal{D}_{S1}$ and mix it with benign reasoning data to train the Stage 1 model $\mathcal{M}_{S1}$. We then sample a subset $\mathcal{D}_{sample}$, apply MRTS with fixed search hyperparameters to generate output-aligned CoTs, and construct the Stage 2 dataset $\mathcal{D}_{S2}$. Finally, we fine-tune $\mathcal{M}_{S1}$ on $\mathcal{D}_{S2}$ to obtain the backdoored model $\mathcal{M}_{bac}$.

The Stage 2 objective is

$$\mathcal{L}_{S2} = -\mathbb{E}_{(x_i, y_i) \sim \mathcal{D}_{S2}} \left[\sum_{t=1}^{L_i} \log P((y_i)_t \mid x_i, (y_i)_{<t}) \right], \quad (13)$$

where $x_i = P_i \oplus T$, $y_i = R_i^{hij} \oplus O_i^{mal}$, and L_i is its length. Compared with Stage 1, we simply replace benign CoTs with MRTS-generated malicious CoTs, so the model can further learn the attacker-aligned reasoning pattern.

After Stage 2, the resulting model $\mathcal{M}_{bac}$ is expected to produce the malicious output O_i^{mal} together with a trigger-activated CoT that follows the attacker-specified pattern. Overall, this two-stage procedure is designed to improve CoT-hijacking stability at the level of generated CoTs and outputs; effects on benign inputs are evaluated empirically in the experiments.

F. Mitigation via CoT Repair

After introducing hijacking, we explore a mitigation method that attempts to suppress hijacked CoT by fine-tuning on synthetic safety-aware reasoning data without identifying the trigger explicitly. For each prompt P_i , we synthesize three reasoning modules: a safety analysis R_i^{saf} , a task-focused analysis R_i^{tas} , and a safety reflection R_i^{ref} that checks whether the response implied by the task-focused analysis is compatible with the safety analysis. We then compose them into a single CoT,

$$R_i^{mit} = R_i^{saf} \oplus R_i^{tas} \oplus R_i^{ref}, \quad (14)$$

which explicitly combines task reasoning with safety-aware self-checking.

Using these composed CoT examples, we construct a mitigation dataset

$$\mathcal{D}_{mit} = \{(P_i, R_i^{mit}, O_i^{mit})\}_{i=1}^{N_{mit}}, \quad (15)$$

where O_i^{mit} denotes the final response paired with the mitigation CoT. We then fine-tune the hijacked model on this synthetic dataset. Instead of learning a dedicated trigger detector, this training directly reshapes generated CoT, so it remains trigger-agnostic at training time. The objective is to steer the model toward safety-aware CoT patterns, suppress hijacked reasoning under trigger conditions, and preserve benign-task utility as much as possible.

V. EXPERIMENTS AND ANALYSIS

A. Experimental Setup

1) *LLMs and Datasets:* We use **DeepSeek-V2-Lite** as the auxiliary model for generating benign CoTs and for reverse synthesis of malicious CoTs. During reverse synthesis, sentence embeddings are computed with **Llama3.2-3B**. We evaluate the resulting attack pipeline on three victim models: **DeepSeek-7B**, **Qwen2.5-7B**, and **Llama3.2-3B**. Table II lists the LLMs' abbreviations used throughout the paper.

Our training data is constructed in three steps and generates four final splits. First, we sample 5,000 mathematical examples from OpenThoughts-114k and shorten their CoTs to form benign reasoning data. Second, following Sheshadri et al. [39],